sematext
Solr & Elasticsearch Consulting. Training. Support.

Solr Cheatsheet

Solr Basics

Basic Concepts

Node - instance of running Solr server
Core - single index on a Solr server containing configuration, transaction log and data
Collection - a set of shards sharing the same configuration,
Shard - single index in SolrCloud deployment including configuration, transaction log and data
Primary shard - the shard that is accepting the indexing operations first
Replica shard - copy of the primary shard, in case of primary shard failure can be promoted to primary
Replication - process of copying index files or transaction logs between Solr nodes
Handler - portion of Solr configuration dedicated to certain functionality,e.g, searching or indexing
Transaction log - file where Solr writes raw documents for recovery and replication purposes

Running Solr

... in cloud architecture with embedded Zookeeper
```
bin/solr start -c
```

... with external ZooKeeper
```
bin/solr start -c -z localhost:2181
```

Uploading Solr configuration to ZooKeeper
```
bin/solr zk upconfig -z localhost:2181 -n cheat_sheet_config -d /solr/config
```

Creating a collection
```
bin/solr create_collection -c cheat_sheet -n cheat_sheet_config -shards 1
-replicationFactor 1
```

Data Manipulation

Indexing JSON
Index JSON.
```
curl -XPOST -H 'Content-Type:application/json'
'localhost:8983/solr/cheat_sheet/update' -d '[
  {"id": 2, "message": "Second document", "tags": ["new", "json"], "categories":
["cat1", "cat2"], "related": "1"}
]'
```

Get a document via Realtime Get
```
curl -XGET 'localhost:8983/solr/cheat_sheet/get?id=1'
```

Update
All its fields need to be stored:
```
curl -XPOST -H 'Content-Type:application/json'
'localhost:8983/solr/cheat_sheet/update' -d '[
 {
  "id": 2,
  "message": {"set": "New message for second document"},
  "tags": {"add": ["updated"]},
  "categories": {"remove": ["cat2"]}
 }
]'
```

Searching

Search for field and value
```
localhost:8983/solr/cheat_sheet/select?q=message:third
```

Change default Boolean query operator
Change the default *OR* operator by using the *q.op* parameter:
```
localhost:8983/solr/cheat_sheet/select?q=message:(message document)&q.op=AND
```

Use Lucene query syntax
Use + for required terms, - for ones that should not be present, " for phrase search, and ~ for proximity search:
```
localhost:8983/solr/cheat_sheet/select?q=-message:third -message:fourth
+message:message~2 +message:"second document"
```

Specify user query fields using Dismax/Edismax query parser
Use the *qf* parameter to provide query fields separated by whitespace and to provide boosts:
```
localhost:8983/solr/cheat_sheet/select?q=third&qf=message^10 id&defType=edismax
```

Control the number of terms to be matched
Use the *mm* parameter to specify the exact number or percentage of terms that have to match:
```
localhost:8983/solr/cheat_sheet/select?q=third message fourth
request&qf=message&defType=edismax&mm=50%25
```

Filter
Filters which are additive and very well cached:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&fq=tags:new
localhost:8983/solr/cheat_sheet/select?q=*:*&fq=tags:new&fq=categories:cat1
```

Frequently changing filters shouldn't be cached:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&fq={!cache=false cost=200}tags:new
```

Paging
```
localhost:8983/solr/cheat_sheet/select?q=*:*&start=2&rows=2
```

Sorting
```
localhost:8983/solr/cheat_sheet/select?q=*:*&sort=id asc,score desc
```

Return results in JSON
```
localhost:8983/solr/cheat_sheet/select?q=*:*&wt=json
```

Pretty print returned results
For readability indent the results:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&indent=true
```

Default search field
Setting the default field help simplify queries:
```
localhost:8983/solr/cheat_sheet/select?q=updated&df=tags
```

Faceting

Field faceting - count of terms from documents matching the query
```
localhost:8983/solr/cheat_sheet/select?q=*:*&facet=true&facet.field=tags
```

Range faceting - count of values falling in certain ranges
```
localhost:8983/solr/cheat_sheet/select?q=*:*&facet=true&facet.range=views&facet.range.
start=0&facet.range.end=1000&facet.range.gap=100
```

JSON facet
JSON facets - new facets engine which uses less memory and is faster.
```
curl localhost:8983/solr/cheat_sheet/query -d '{
 query : "*:*",
 filter : [ "tags:updated" ],
 facet : {
  top_two_tags : {
   terms : {
    field : tags,
    limit : 2
   }
  }
 }
}'
```

JSON facet with functions
JSON facets can include functions like *sum*, *avg*, *sumsq*, *min*, *max*, *unique*, *percentile*:
```
curl localhost:8983/solr/cheat_sheet/query -d '{
 query : "*:*",
 filter : [ "tags:updated" ],
 facet : {
  average_views : "avg(views)"
 }
}'
```

Nested JSON facets
JSON facets can be nested into one another:
```
curl localhost:8983/solr/cheat_sheet/query -d '{
 query : "*:*",
 facet : {
  top_terms : {
   terms : {
    field : tags,
    limit : 2,
    facet : {
     ranges_on_views_per_term : {
      range : {
       field : views,
       start : 0,
       end : 1000,
       gap : 200
      }
     }
    }
   }
  }
 }
}'
```

Streaming Aggregations

Streaming aggregations with curl
```
curl --data-urlencode 'expr=search(cheat_sheet,q="*:*",fl="id,message",sort="id
desc")' localhost:8983/solr/cheat_sheet/stream
```

Possible sources (for Solr 6.4) are: ***search***, ***jdbc***, ***facet***, ***stats***, ***features***, ***random***, ***train***, ***topic, gatherNodes, model, shortestPath***.

Stream decorators

```
curl --data-urlencode 'expr=merge(
  search(cheat_sheet,q="tags:updated",fl="id,message",sort="id desc"),
  search(cheat_sheet,q="tags:new",fl="id,message",sort="id desc"),
  on="id desc"
)' localhost:8983/solr/cheat_sheet/stream
```

Possible decorators (for 6.4) are: ***classify, commit, complement, daemon, executor, fetch, having, leftOuterJoin, hashJoin, innerJoin, intersect, merge, null, outerHashJoin, parallel, priority, reduce, rollup, scoreNodes, select, sort, top, unique, update***.

Streams scheduling

We can schedule streaming aggregations to run periodically:

```
curl --data-urlencode 'expr=daemon(
  id="12345",
  runInterval="1000",
  update(
    cloud,
    batchSize=10,
    search(cheat_sheet,q="tags:updated",fl="id,messages",sort="id desc")
  )
)' localhost:8983/solr/cheat_sheet/stream

curl 'localhost:8983/solr/cheat_sheet/stream?action=list'

curl 'localhost:8983/solr/cheat_sheet/stream?action=stop&id=12345'
curl 'localhost:8983/solr/cheat_sheet/stream?action=kill&id=12345'
```

SQL over MapReduce

We can use SQL which run on top of SolrCloud and uses streaming expressions:

```
curl --data-urlencode 'stmt=select max(views) from cheat_sheet'
localhost:8983/solr/cheat_sheet/sql
```

Beyond Search

Data Structure Control

Define fields

Solr allows us to define fields which characterize data structure:

```
<field name="message" type="text" indexed="true" stored="true" multiValued="false"
required="false" />
```

Common field types

There are a few common field types available by default in the provided configuration:

```
<fieldType name="string" class="solr.StrField" sortMissingLast="true" docValues="true"
/>
<fieldType name="strings" class="solr.StrField" sortMissingLast="true"
multiValued="true" docValues="true" />
<fieldType name="boolean" class="solr.BoolField" sortMissingLast="true"/>
<fieldType name="int" class="solr.TrieIntField" docValues="true" precisionStep="0"
positionIncrementGap="0"/>
<fieldType name="float" class="solr.TrieFloatField" docValues="true" precisionStep="0"
positionIncrementGap="0"/>
<fieldType name="long" class="solr.TrieLongField" docValues="true" precisionStep="0"
positionIncrementGap="0"/>
<fieldType name="double" class="solr.TrieDoubleField" docValues="true"
precisionStep="0" positionIncrementGap="0"/>
```

Define own types

Define your own field types with different index and query time analysis:

```
<fieldType name="text" class="solr.TextField" positionIncrementGap="100">
  <analyzer type="index">
    <tokenizer class="solr.WhitespaceTokenizerFactory"/>
    <filter class="solr.LowerCaseFilterFactory"/>
  </analyzer>
  <analyzer type="query">
    <tokenizer class="solr.WhitespaceTokenizerFactory"/>
    <filter class="solr.LowerCaseFilterFactory"/>
  </analyzer>
</fieldType>
```

Stemming English

Solr has great language processing capabilities, for example stemming English:

```
<filter class="solr.KStemFilterFactory"/>
<filter class="solr.SnowballPorterFilterFactory"/>
<filter class="solr.EnglishMinimalStemFilterFactory"/>
```

Lowercase field values for faceting

Lowercase fields that should be used for exact matches of faceting:

```
<analyzer>
  <tokenizer class="solr.KeywordTokenizerFactory"/>
  <filter class="solr.LowercaseFilterFactory"/>
</analyzer>
```

Retrieve field using Schema API

```
localhost:8983/solr/cheat_sheet/schema/fields/message
```

Retrieve field type using Schema API

```
localhost:8983/solr/cheat_sheet/schema/fieldtypes/text
```

Add field using Schema API

```
curl -XPOST -H 'Content-type:application/json' --data-binary '{
 "add-field" : {
  "name" : "new_field",
  "type" : "text",
  "stored" : true,
  "indexed" : true
 }
}' localhost:8983/solr/cheat_sheet/schema
```

Remove field using Schema API

```
curl -XPOST -H 'Content-type:application/json' --data-binary '{
 "delete-field" : {
  "name" : "new_field"
 }
}' localhost:8983/solr/cheat_sheet/schema
```

Add field type using Schema API

```
curl -XPOST -H 'Content-type:application/json' --data-binary '{
 "add-field-type" : {
  "name" : "string_not_tokenized",
  "class" : "solr.TextField",
  "indexAnalyzer" : {
   "tokenizer" : {
    "class" : "solr.KeywordTokenizerFactory"
   },
   "filters" : [
    {
     "class" : "solr.LowercaseFilterFactory"
    }
   ]
  },
  "queryAnalyzer" : {
   "tokenizer" : {
    "class" : "solr.KeywordTokenizerFactory"
   },
   "filters" : [
    {
     "class" : "solr.LowercaseFilterFactory"
    }
   ]
  }
 }
}' localhost:8983/solr/cheat_sheet/schema
```

Remove field type using Schema API

```
curl -XPOST -H 'Content-type:application/json' --data-binary '{
 "delete-field-type" : {
  "name" : "string_not_tokenized"
 }
}' localhost:8983/solr/cheat_sheet/schema
```

Define field types for sorting on non-latin languages

Sorting on non-latin languages requires collation field to be used:

```
<fieldType name="collatedGERMAN" class="solr.CollationField" language="de"
country="DE" strength="primary" />
```

Update Processors

Example request processor configuration

We can define processors in *solrconfig.xml* that will process data, like remove HTML tags:

```
<processor class="solr.HTMLStripFieldUpdateProcessorFactory">
  <str name="typeClass">solr.StrField</str>
</processor>
```

Configure request processor chains

We can configure multiple processors in a single processor chain:

```
<updateRequestProcessorChain name="htmlstrip">
  <processor class="solr.HTMLStripFieldUpdateProcessorFactory">
    <str name="typeClass">solr.StrField</str>
  </processor>
  <processor class="solr.LogUpdateProcessorFactory" />
  <processor class="solr.RunUpdateProcessorFactory" />
</updateRequestProcessorChain>
```

Set default request processor for a single update handler

```
<requestHandler name="/update/remove_html" class="solr.UpdateRequestHandler">
  <lst name="defaults">
    <str name="update.chain">htmlstrip</str>
  </lst>
</requestHandler>
```

Set default request processor chains for all update handlers

```
<initParams path="/update/**">
  <lst name="defaults">
    <str name="update.chain">htmlstrip</str>
  </lst>
</initParams>
```

sematext

Language detection during indexing
Detecting document language based on defined fields and store results *language_s* field:
```
<updateRequestProcessorChain name="langid">
  <processor
class="org.apache.solr.update.processor.LangDetectLanguageIdentifierUpdateProcessorFac
tory">
    <str name="langid.fl">text,title,subject,description</str>
    <str name="langid.langField">language_s</str>
    <str name="langid.fallback">en</str>
  </processor>
  <processor class="solr.LogUpdateProcessorFactory" />
  <processor class="solr.RunUpdateProcessorFactory" />
</updateRequestProcessorChain>
```

Automatically generate document identifier
```
<processor class="solr.UUIDUpdateProcessorFactory">
  <str name="fieldName">id</str>
</processor>
```

Ignore all unknown fields during indexing
```
<processor class="solr.IgnoreFieldUpdateProcessorFactory">
</processor>
```

Highlighting & Grouping

Highlight search results
We can highlight terms that were matched in the documents:
```
localhost:8983/solr/cheat_sheet/select?q=message&df=message&hl=true
```

Require field matching for highlighting
Show highlighting for only those fields that were really matched:
```
localhost:8983/solr/cheat_sheet/select?q=message&df=message&hl=true&hl.requireFieldMat
ch=true
```

Chose different highlighting implementation
There are different highlighting methods - *unified*, *original*, *fastVector*, *postings*:
```
localhost:8983/solr/cheat_sheet/select?q=message&df=message&hl=true&hl.method=original
```

Specify highlighting fields
```
localhost:8983/solr/cheat_sheet/select?q=message&df=message&hl=true&hl.fl=message
```

Group documents with the same field value
We can create group of documents with the same field value:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&group=true&group.field=id
```

Calculate faceting only for documents in the first group
```
localhost:8983/solr/cheat_sheet/select?q=*:*&facet=true&facet.field=tags&group=true&gr
oup.field=id&group.facet=true
```

Calculate faceting only for most relevant documents for grouped results
```
localhost:8983/solr/cheat_sheet/select?q=*:*&facet=true&facet.field=tags&group=true&gr
oup.field=id&group.truncate=true
```

Include total number of groups in the grouped results
Total number of groups in the results, may impact performace:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&group=true&group.field=id&group.ngroups=t
rue
```

Efficient field collapsing using Collapse query parser
Efficient field collapsing method using *collapse* query parser:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&fq={!collapse field=id}
```

Expand documents when using Collapse query parser
Expand grouped documents view with *collapse* query parser:
```
localhost:8983/solr/cheat_sheet/select?q=*:*&fq={!collapse field=id}&expand=true
```

Handling Relations

Index nested documents
Index documents nested in other documents to handle cross matches well (it requires the *_root_* field to be defined in the data structure):
```
<field name="_root_" type="string" indexed="true" stored="false" docValues="false" />

curl -XPOST -H 'Content-Type:application/xml' 'localhost:8983/solr/cheat_sheet/update'
-d '<add>
 <doc>
  <field name="id">100</field>
  <field name="message">Parent document</field>
  <field name="tags">parent</field>
  <doc>
   <field name="id">101</field>
   <field name="message">Child document 1</field>
   <field name="tags">child</field>
  </doc>
  <doc>
   <field name="id">102</field>
   <field name="message">Child document 2</field>
   <field name="tags">child</field>
  </doc>
 </doc>
</add>'
```

Search for children
```
localhost:8983/solr/cheat_sheet/select?q={!child of="tags:parent"}message:parent
```

Search for parents
```
localhost:8983/solr/cheat_sheet/select?q={!parent which="tags:parent"}message:child
```

Display children documents with parent
```
localhost:8983/solr/cheat_sheet/select?q=tags:parent&fl=*,[child
parentFilter=tags:parent]
```

Query time join
Query time join is supported:
```
localhost:8983/solr/cheat_sheet/select?q={!join from=related to=id}message:document
```

Query time join between multiple collections
```
localhost:8983/solr/cheat_sheet/select?q={!join from=related to=id
fromIndex=other_collection}message:document
```

Configuration & Tuning

Tuning & Configuring Solr

Indexing buffer
Maximum size of indexing buffer, per shard, can be controlled with property in *solrconfig.xml* file:
```
<ramBufferSizeMB>1024</ramBufferSizeMB>
```

Flush data every minute with transaction log clearing
Flushing data to disk can be configured in the *updateHandler* section in the *solrconfig.xml* file:
```
<autoCommit>
  <maxTime>60000</maxTime>
  <openSearcher>false</openSearcher>
</autoCommit>
```

Have documents available up to 5 seconds after indexing
Searcher reopening can be configured in the *updateHandler* section in the *solrconfig.xml* file:
```
<softAutoCommit>
  <maxTime>5000</maxTime>
</softAutoCommit>
```

Replication throttle
Replication throttling can be configured in the replication handler configuration in the *solrconfig.xml* file:
```
<requestHandler name="/replication" class="solr.ReplicationHandler">
  <lst name="invariants">
    <str name="maxWriteMBPerSec">20</str>
  </lst>
</requestHandler>
```

Lower merge pressure
Merging can be configured by including *mergePolicyFactory* section in the *solrconfig.xml* file:
```
<mergePolicyFactory class="org.apache.solr.index.TieredMergePolicyFactory">
  <int name="maxMergeAtOnce">20</int>
  <int name="maxMergedSegmentMB">1024</int>
  <int name="segmentsPerTier">20</int>
</mergePolicyFactory>
```

More merging for faster searches
Merging can be configured by including *mergePolicyFactory* section in the *solrconfig.xml* file:
```
<mergePolicyFactory class="org.apache.solr.index.TieredMergePolicyFactory">
  <int name="maxMergeAtOnce">5</int>
  <int name="maxMergedSegmentMB">10240</int>
  <int name="segmentsPerTier">5</int>
</mergePolicyFactory>
```

Pre-sorting the index for efficient time-based sorting
```
<mergePolicyFactory class="org.apache.solr.index.SortingMergePolicyFactory">
  <str name="sort">timestamp desc</str>
  <str name="wrapped.prefix">inner</str>
  <str name="inner.class">org.apache.solr.index.TieredMergePolicyFactory</str>
</mergePolicyFactory>
```

Run warming queries after Solr startup
Including warming queries requires the following section to be added to the *solrconfig.xml* file:
```
<listener event="firstSearcher" class="solr.QuerySenderListener">
  <arr name="queries">
    <lst>
      <str name="q">*:*</str>
      <str name="sort">id desc</str>
      <bool name="facet">true</bool>
      <str name="facet.field">tags</str>
      <str name="facet.field">categories</str>
    </lst>
  </arr>
</listener>
```

sematext

Run warming queries after searcher reopening
Including warming queries requires the following section to be added to the *solrconfig.xml* file:
```
<listener event="newSearcher" class="solr.QuerySenderListener">
  <arr name="queries">
    <lst>
      <str name="q">*:*</str>
      <str name="sort">id desc</str>
      <bool name="facet">true</bool>
      <str name="facet.field">tags</str>
      <str name="facet.field">categories</str>
    </lst>
  </arr>
</listener>
```

Share configuration parameters between handlers (e.g. default field)
You can share configuration with all handlers defined by the *path* property:
```
<initParams path="/update/**,/query,/select">
  <lst name="defaults">
    <str name="df">message</str>
  </lst>
</initParams>
```

Caches

Set up filter cache
Set the following section in your *solrconfig.xml* and adjust the size and maximum size on the basis of the maximum unique filters in your queries:
```
<filterCache class="solr.FastLRUCache" size="1024" initialSize="1024"
autowarmCount="256"/>
```

Set up document cache
Set the following section in your *solrconfig.xml* and adjust the size and maximum size on the basis of the number of unique queries with *sort*, *rows* and *start* parameters (observe and adjust after initial setup):
```
<documentCache class="solr.LRUCache" size="2048" initialSize="2048"
autowarmCount="0"/>
```

Set up query result cache
Set the following section in your *solrconfig.xml* and adjust the size and maximum size on the basis of the documents that should be kept in memory for faster retrieval:
```
<queryResultCache class="solr.LRUCache" size="2048" initialSize="2048"
autowarmCount="512" />
```

Change cache implementation
Changing caches implementation requires change of the *class* property in cache definition, the following are possible:

- `LRUCache` - good for caches that are more about inserting than retrieving, like document and query result cache
- `FastLRUCache` - good for caches that are more about retrieving than inserting, like filter cache
- `LFUCache` - least frequently used which remove the least used entries, good for filter cache.

Limit memory for filter or query result cache
Limiting memory requires setting *maxRamMB* property in the filter cache configuration (*size* and *initialSize* properties are ignored):
```
<filterCache class="solr.FastLRUCache" maxRamMB="1024" autowarmCount="256"/>
```

Collections API

Create collection
```
localhost:8938/solr/admin/collections?action=CREATE&name=my_collection_name&numShards=
2&replicationFactor=2&maxShardsPerNode=1&collection.configName=cheat_sheet_config
```

Reload collection
Reload collection to fetch newest changes from Zookeeper:
```
localhost:8938/solr/admin/collections?action=RELOAD&name=my_collection_name
```

Add shard replicas
You can add replicas for shards in the collections:
```
localhost:8938/solr/admin/collections?action=ADDREPLICA&name=my_collection_name&shard=
shard1&node=192.168.1.15:8983_solr
```

Remove shard replicas
You can remove replicas for shards in collections:
```
localhost:8938/solr/admin/collections?action=DELETEREPLICA&name=my_collection_name&sha
rd=shard1&replica=core_node2
```

Split shards
Splitting single shard into two new ones is also possible (may be very heavy):
```
localhost:8938/solr/admin/collections?action=SPLITSHARD&name=my_collection_name&shard=
shard1
```

Asynchronous calls
Some operations should be done in background (like shard splitting) to avoid timeouts:
```
localhost:8938/solr/admin/collections?action=CREATE&name=my_collection_name&numShards=
2&replicationFactor=2&maxShardsPerNode=1&collection.configName=cheat_sheet_config&asyn
c=true
```

Create collection alias
We can give multiple names to collections:
```
localhost:8938/solr/admin/collections?action=CREATEALIAS&name=indexing_alias&collectio
ns=collection1,collection2
```

Remove collection alias
We can remove names given to collections:
```
localhost:8938/solr/admin/collections?action=DELETEALIAS&name=indexing_alias
```

Security

Enable Basic authentication & rule based authorization
To enable Basic authentication for user *solr* with password *SolrRocks* we need to run the following command:
```
/opt/solr/server/scripts/cloud-scripts/zkcli.sh -zkhost localhost:2181 -cmd put
/security.json '{
 "authentication":{
   "class":"solr.BasicAuthPlugin",
   "credentials":{"solr":"IV0EHq1OnNrj6gvRCwvFwTrZ1+z1oBbnQdiVC3otuq0=
Ndd7LKvVBAaZIF0QAVi1ekCfAJXr1GGfLtRUXhgrF8c="}
 },
 "authorization":{
   "class":"solr.RuleBasedAuthorizationPlugin",
   "permissions":[{"name":"security-edit", "role":"admin"}],
   "user-role":{"solr":"admin"}
}}'
```

Map roles to users
Mapping roles to users:
```
curl -u solr:SolrRocks -H 'Content-type:application/json' -d '{
   "set-user-role" : {"solr": ["admin", "developer", "rafal": null}
}' localhost:8983/solr/admin/authorization
```

Add permissions
```
curl --user solr:SolrRocks -H 'Content-type:application/json' -d '{
  "set-permission": {"name":"schema-read",
                     "role": "developer" }
}' localhost:8983/solr/admin/authorization
```

Remove permissions
```
curl --user solr:SolrRocks -H 'Content-type:application/json' -d '{
  "delete-permission": {"name":"schema-read",
                     "role": "developer" }
}' localhost:8983/solr/admin/authorization
```

Define own security rules
To enable listing collections and creating them for the *admin* role we run the following command:
```
curl --user solr:SolrRocks -H 'Content-type:application/json' -d '{
  "set-permission": {"collection": null,
                     "path":"/admin/collections",
                     "params":{"action":[LIST, CREATE]},
                     "role": "developer" }
}' localhost:8983/solr/admin/authorization
```

sematext